SPAR: Personalized Content-Based Recommendation via Long Engagement Attention

Chiyu Zhang[ξ,λ,⋆] **Yifei Sun**[λ] **Jun Chen**[λ] **Jie Lei**[λ] **Muhammad Abdul-Mageed**[ξ,γ]
Sinong Wang[λ] **Rong Jin**[λ] **Sem Park**[λ] **Ning Yao**[λ] **Bo Long**[λ]
[ξ] The University of British Columbia [λ] Meta AI [γ]Department of NLP & ML, MBZUAI
chiyuzh@mail.ubc.ca {sunyifei,junchen20,jielei}@meta.com

arXiv:2402.10555v2 [cs.IR] 22 May 2024

Abstract

Leveraging users' long engagement histories is essential for personalized content recommendations. The success of pretrained language models (PLMs) in NLP has led to their use in encoding user histories and candidate items, framing content recommendations as textual semantic matching tasks. However, existing works still struggle with processing very long user historical text and insufficient user-item interaction. In this paper, we introduce a content-based recommendation framework, SPAR, which effectively tackles the challenges of holistic user interest extraction from the long user engagement history. It achieves so by leveraging PLM, poly-attention layers and attention sparsity mechanisms to encode user's history in a session-based manner. The user and item side features are sufficiently fused for engagement prediction while maintaining standalone representations for both sides, which is efficient for practical model deployment. Moreover, we enhance user profiling by exploiting large language model (LLM) to extract global interests from user engagement history. Extensive experiments on two benchmark datasets demonstrate that our framework outperforms existing state-of-the-art (SoTA) methods.

1 Introduction

With the prosperity of the digital world, billions of users engage daily with various digital content, including news, social media posts, and online books. Content-based recommendation systems (Gu et al., 2016; Okura et al., 2017; Malkiel et al., 2020; Mao et al., 2023) utilize textual contents (e.g., news, books) and the textual sequence of a user's engagement history to recommend more precise, relevant, and personalized content. News services, such as Google News[1] and MSN,[2] provide personalized news articles based on a user's browsing history. Platforms like Reddit[3] and X (formerly Twitter)[4] enable users to explore and engage with posts and threads of interest. Goodreads offers book recommendations based on a user's review history.[5]

Content-based recommendation systems can alleviate long-tailed and cold-start problems in traditional ID-based recommendation systems by learning the transferable semantic meanings of text content (Wu et al., 2021a; Liu et al., 2023a). Hence, a crucial component in a content-based recommendation system is the content encoder, employed to encode textual content into latent space and capture useful information for the task. Early works use pretrained word embeddings to initialize the embedding layer and train newly initialized convolutional neural networks (CNN) (An et al., 2019), recurrent neural networks (RNN) (Okura et al., 2017), or attention layers (Wu et al., 2019a) to further aggregate the contents. Recently, PLMs have revolutionized the protocol of NLP and demonstrated their superiority across a wide range of tasks (Touvron et al., 2023a; OpenAI, 2023; Wu et al., 2023; Lyu et al., 2023; Wu and Aji, 2023; Anil et al., 2023). Many studies (Wu et al., 2021c; Li et al., 2022; Liu et al., 2023c,a) have incorporated PLMs into recommendation systems to encode textual inputs. While these studies have shown success in content-based recommendation, they still face challenges in encoding long user engagement histories. For a typical recommendation system like Google News, a user's engagement history usually contains >50 news. Since each piece of news has roughly 128 tokens as its brief content, compiling all engaged news into one text field can yield se-

⋆ Work done during Meta internship.

[1] https://news.google.com/
[2] https://www.msn.com/en-us/news
[3] https://www.reddit.com/
[4] https://twitter.com/
[5] https://www.goodreads.com/

quences over 5K tokens. Due to this very long sequence length, the common practice in previous works is to encode each history content separately and fuse them later. The representation of the first token (i.e., '<SOS>' token) is usually used as the sequence-level embedding, which may fail to encapsulate fine-grained information. Separately encoding historical contents alleviates the memory issue in self-attention but lacks cross-content interactions. To address this limitation, we introduce a sparse attention mechanism for encoding session-based user behavior (Liu et al., 2023d) and employ a poly-attention (a.k.a. codebook-based attention) module for long history integration,[6] balancing comprehensive information fusion and moderate computational demands.

Recent research (Qi et al., 2022; Li et al., 2022; Xu et al., 2023) explores the early fusion of user and candidate item information to improve their interactions and click-through rate (CTR) prediction accuracy, yet this results in user/candidate representations that are dependent on each other and cannot be pre-computed. Standalone user and candidate item embeddings are important for both the lightweight retrieval stage (e.g., using similarity between pre-computed embeddings) and the ranking stage, as input for pre-computed upstream features. To overcome this challenge, we propose a novel framework that ensures standalone representations for both user and candidate items while capturing fine-grained features and enriching user-item interactions. Inspired by the concept of *post-fusion* (Gong et al., 2020; Izacard and Grave, 2021), we employ poly-attention (Humeau et al., 2019) to globally aggregate user history. To further reduce computation complexity and prevent a dramatic increase in the entropy of the attention distribution (Han et al., 2023) in cases of very long sequences, we introduce three attention strategies to our poly-attention module: local sliding-window attention, global attention, and random attention. To enrich the post-interaction between a user and candidate item, we implement additional poly-attention layers to obtain multiple embeddings for both user and candidate sides. Furthermore, we enhance the user history by utilizing an LLM as a user-interest profiler to extract the user's global interests from their engagement history.

Our main contributions are summarized as follows:

(1) We propose a framework for *post-fusion* with **S**parse **P**oly-**A**ttention for content **R**ecommendation (SPAR), that incorporates multiple *poly-attention* layers and *sparse attention* mechanisms to hierarchically fuse token level embeddings of session-based user history texts by PLM. SPAR effectively extracts user-interest embeddings from long history text sequences and enables sufficient interaction between user and candidate item(Section 3).

(2) We demonstrate the effectiveness of SPAR by testing on two widely used datasets. Our approach surpasses the SoTA methods, achieving a significant improvement of 1.48% and 1.15% in AUC scores for the MIND news recommendation and Goodreads book recommendation datasets, respectively (Section 5.1).

(3) Our extensive ablation studies demonstrate the impact of each component within our framework, offering insights into potential trade-offs for designing a content-based recommendation system (Section 5.2).

2 Related Work

Recommendation systems utilize user engagement history to identify items of potential interest to a user. Traditional recommendation systems record user engagement as a sequential list of item IDs and learn to recommend items using Markov chain assumptions (He and McAuley, 2016) or matrix factorization methods (Rendle, 2010). However, these ID-based methods struggle with challenges like the prevalence of long-tailed items and the "cold start" problem, where numerous new items continually emerge (Schein et al., 2002; Lam et al., 2008). Recently, several studies (Cui et al., 2022; Li et al., 2022; Liu et al., 2023d) have sought to mitigate these issues by introducing content-based recommendations. In these systems, user and candidate item features are described in text, this makes the recommendation systems consume pure textual features as input. The user and candidate representations are learned by various neural networks, including CNNs (An et al., 2019), RNNs (Okura et al., 2017; Wang et al., 2022), and attention mechanisms (Wu et al., 2019b,c; Qi et al., 2021).

[6] In this paper, the terms 'codebook-based attention' and 'poly-attention' are used interchangeably.

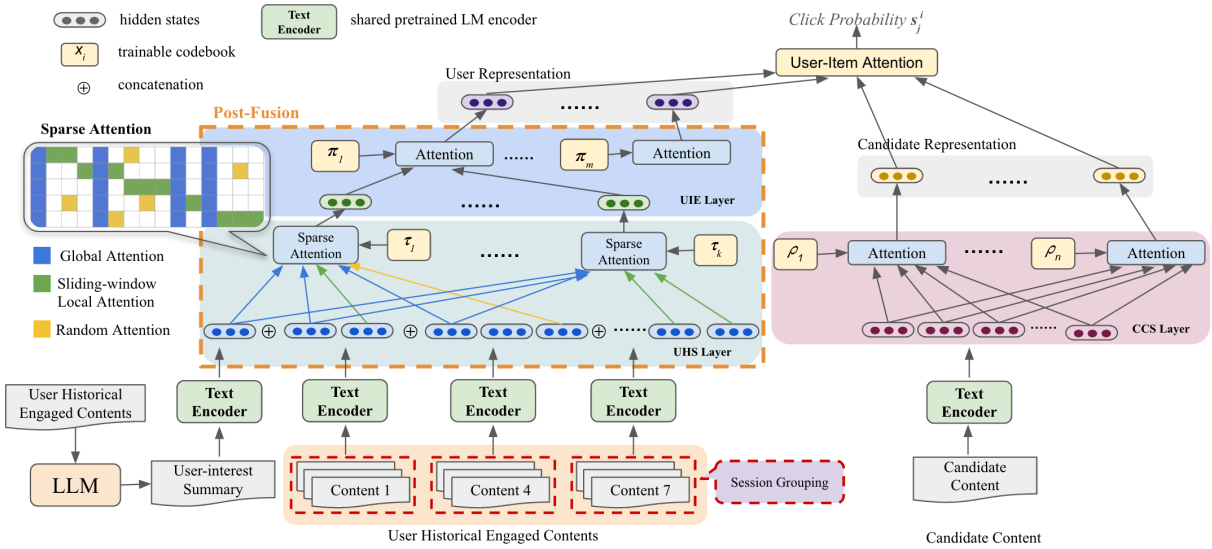

Figure 1: Overview of our framework, SPAR.

Early content-based recommendation models (Wu et al., 2019c; An et al., 2019) employed pretrained word embeddings, such as GloVe (Pennington et al., 2014), to initialize their embedding weights. Recent studies have been significantly bolstered by the use of PLMs. Wu et al. (2021a) introduced a BERT-based two-tower framework to encode user history and candidate content, where the cosine similarity between <user, candidate> embedding pair is used as relevance score. Bi et al. (2022) enhanced the encoder's capacity to extract effective information from news by introducing two auxiliary tasks: category classification and named entity recognition. Liu et al. (2022, 2023b) did further in-domain pretraining to adapt the generic pretrained encoders. Li et al. (2022) argued that a single representation vector is insufficient for capturing a user's diverse interests, and thus introduced a codebook-based attention network to obtain multiple user interest vectors. However, these studies typically encode content items individually and rely solely on the hidden state of the '[CLS]' token to represent each item. We hypothesize that this approach lacks fine-grained token-level signals which are crucial to digest the complex interaction among long sequence of content engagement history, as we show in our ablation studies (Section 5.2).

On another aspect, several studies (Qi et al., 2022; Li et al., 2022; Xu et al., 2023) have introduced candidate-aware user encoder to enhance interaction and fusion between the user-side and candidate-side. Li et al. (2022) introduced a category-aware attention weighting strategy that re-weights historical news based on their category similarity to the candidate news. Mao et al. (2023) proposed an encoder-decoder framework, encoding the historical contents with a BART encoder (Lewis et al., 2020), using the candidate content as input for the decoder, and measuring the user-item relevance score by calculating the perplexity of the candidate content at the decoder. However, these approaches do not support pre-computation of user embeddings due to their dependence on awareness of the candidate item. Real-world applications require maintaining standalone representations for both users and candidate items to handle billion-scale daily interactions and support large-scale retrieval. Hence, our SPAR is designed to maintain independent representations, while extracting fine-grained feature sets from long history sequence with computational efficiency.

3 Methodology

We now introduce our proposed framework, SPAR. As illustrated in Figure 1, SPAR incorporates session-based attention sparsity to encode a user's long history using a PLM. We then employ a user history summarizing (UHS) layer followed by a user interest extracting (UIE) layer to derive a comprehensive user representation (Section 3.2). To further enrich the user-side representation, we utilize an LLM to generate summaries of user interests based on their engagement history (Section 3.2). For candidate content, we apply the shared PLM and candidate content summarizing (CCS) layer to obtain its embeddings (Section 3.3). In the end, a

lightweight attention layer is used to interact user and candidate content embeddings for CTR prediction (Section 3.4).

3.1 Problem Formulation

We focus on content-based recommendation tasks that solely utilize natural language text as input. Given a user u_i and a candidate content (e.g., news or book) $\bar{e}_j$, our objective is to derive a relevance score s_j^i, indicating how likely the user u_i will engage with (e.g., click) the content $\bar{e}_j$. Considering a set of candidate contents $C = \{\bar{e}_1, \bar{e}_2, \ldots, \bar{e}_j\}$, these contents are ranked based on their relevance scores $\{s_1^i, s_2^i, \ldots, s_j^i\}$ for user u_i, with the high scored contents displayed to user u_i in the service. Therefore, it is critical to effectively extract user interests from their engagement history. The user u_i is characterized by a sequence of k historically engaged contents (such as browsed news or positively rated books) $E_{u_i} = \{e_1^i, e_2^i, \ldots, e_k^i\}$, sorted descendingly by engagement time. Each content e comprises text information, including title, abstract, and category. In practical applications, it is expected to pre-compute standalone embeddings for both user u_i and candidate content $\bar{e}_j$ prior to calculating the relevance score. The embeddings of user u_i and candidate content $\bar{e}_j$ are represented as $\Gamma_i \in \mathrm{R}^{m \times d}$ and $\Lambda_i \in \mathrm{R}^{n \times d}$, respectively, where $m, n \ll d$ and $d \ll J$, with J representing the total number of candidate contents.

3.2 User History Encoder

Session-Based PLM Encoding For each content, we combine its multiple fields into a single text sequence. For instance, a news item is compiled using the template: "News Title: $\langle title \rangle$; News Abstract: $\langle abstract \rangle$; News Category: $\langle category \rangle$". On the user side, we concatenate the engagement history $E = \{e_1, e_2, \ldots, e_k\}$ into a long sequence, adding a start-of-sentence '$\langle SOS \rangle$' and an end-of-sentence '$\langle EOS \rangle$' token at the beginning and end of each content, respectively, to denote the content item boundary. To obtain representation embeddings for users and candidate items, we employ a PLM $\Phi(\cdot)$ for encoding. The encoder is shared between the user and the candidate sides. However, the user-side content history is often much longer (exceeding 5K tokens) than the maximum length capacity of PLMs (e.g., 512 tokens for RoBERTa (Liu et al., 2019)), leading to a substantial memory requirement due to the quadratic complexity of self-attention. To mitigate this, we employ an *attention sparsity mechanism* (Liu et al., 2023d), encoding session-based user behavior by grouping the user engagement history into g subsequences $E = \{\eta_1, \eta_2, \ldots, \eta_g\}$. Each session η_1 contains all l tokens from p contents, i.e., $\eta_1 = \{e_1, e_2, \ldots, e_p\} = \{w_1, w_2, \ldots, w_l\}$, which can reflect the user's interests in a specific period and enhance in-session interactions. These subsequences are encoded separately by encoder Φ:

$$h_{\cdot}^g = \Phi(\eta_g), \tag{1}$$

where $h_{\cdot}^g \in \mathrm{R}^{l \times d}$, d is the hidden dimension of encoder Φ, and h_1^g corresponds to the hidden states of input token w_1 in η_g. The attention sparsity avoids the self-attention calculations between different sessions in Transformer encoder, thus lowering the computational complexity of encoding from $O(n^2)$ to $O(n)$. In previous work (Li et al., 2022; Wu et al., 2021a), the hidden state of the first token was utilized as the representation for the entire sequence. However, this approach may not be sufficient to capture fine-grained information pertinent to user interests for a long sequence. Therefore, we take token level embedding in each subsequence and then concatenate the hidden states of all subsequences, $H = \left[h_{\cdot}^1 \oplus h_{\cdot}^2 \oplus \cdots \oplus h_{\cdot}^g\right]$ where $H \in \mathrm{R}^{L \times d}$ and L is the total number of tokens in engaged contents of a user, which can be more than 5K tokens.

LLM User Interest Summary Recently, LLMs (Touvron et al., 2023b; Anil et al., 2023; Wang et al., 2024; Wu et al., 2023) have demonstrated remarkable capabilities in general question answering, long-sequence summarization, and reasoning. In light of this, we enrich user history by employing LLM as user-interest summarizers to extract a user's global interests. We prompt LLM to generate summaries of user interests in natural language, drawing on their engagement history. These generated summaries, denoted as η_+, are then encoded by encoder Φ. We prepend the sequence of hidden states $h_{\cdot}^+$ to H, resulting in an augmented sequence $H^+ \in \mathrm{R}^{L^+ \times d}$, where L^+ represents the total length of the sequence after adding LLM's user-interest summary.

User History Summarizing (UHS) While session-based encoding enables the model to capture a user's local interests within a specific period, characterizing global interests across sessions is also crucial. Inspired by the concept of post-

fusion (Izacard and Grave, 2021), we propose employing a poly-attention layer (Humeau et al., 2019) to globally summarize k user-engaged contents into k embeddings using H^+. This enables a comprehensive representation of user history, integrating both local and global perspectives. We learn k context codes (i.e., $\tau_1, \tau_2, \ldots, \tau_k$), where each $\tau_a \in \mathrm{R}^{1\times p}$ is designed to learn a contextual representation y_a by attending over all L^+ tokens in H^+. The hidden dimension of code τ_a is p, which is smaller than d to reduce compute cost. Each user-engaged content embedding y_a is computed as follows:

$$y_a = W^{\tau_a} H^+, \tag{2}$$

where $W^{\tau_a} \in \mathrm{R}^{1\times L^+}$ are attention weights across L^+ tokens in H^+ and are calculated as follows:

$$W^{\tau_a} = \mathrm{softmax}\left[\tau_a \mathrm{tanh}(H^+ W^c)^\top\right], \tag{3}$$

where τ_a and $W^c \in \mathrm{R}^{d\times p}$ are trainable parameters.

Sparse Attention in UHS While the introduction of codebook queries significantly reduces the size of queries, the exceedingly long sequence of keys can still lead to the entropy of attention distribution increasing dramatically (Han et al., 2023) and significant compute complexity. To address this, we constrain each context code τ_a to attend only to a limited range of tokens. Specifically, each context code τ_a attends to a subset of $\hat{\mathcal{L}}$ tokens, rather than all L^+ tokens, where $\hat{\mathcal{L}} \ll L^+$. Inspired by Zaheer et al. (2020), we implement this with three attention mechanisms, namely, *local window attention*, *global attention*, and *random attention*, to capture both local and global information from the user's engagement history. For local attention, we set a sliding window that specifies an attention range for each context code. To provide global information across the whole user history, we make the positions of all '$\langle SOS \rangle$' tokens globally visible. Additionally, we randomly select 10% of the remaining tokens to be visible to code τ_a.

User Interests Extracting (UIE) Upon obtaining these k representations of user interaction history, we concatenate them as $Y = [y_1 \oplus y_2 \oplus \cdots \oplus y_k]$, and then apply another poly-attention layer to extract users' interests. Specifically, we introduce m context codes (i.e., $\pi_1, \pi_2, \ldots, \pi_m$) to represent the overall interests of a user as m d-dimensional vectors. Each user-interest vector ψ_a is calculated as follows:

$$\psi_a = \mathrm{softmax}\left[\pi_a \mathrm{tanh}(Y W^f)^\top\right] Y, \tag{4}$$

where $\pi_a \in \mathrm{R}^{1\times p}$ and $W^f \in \mathrm{R}^{d\times p}$ are trainable parameters. We then combine m user interest vectors to $\Gamma \in \mathrm{R}^{m\times d}$ as the user-side representation.

3.3 Candidate Content Encoder

To encode a candidate content, we utilize the shared encoder Φ. The candidate item is constructed using the same template as outlined in Section 3.2.

Candidate Content Summarizing (CCS) Differing from previous studies (Li et al., 2022; Wang et al., 2022) that represent candidate content solely by the first token of the sequence, we introduce n context codes (i.e., $\rho_1, \rho_2, \ldots, \rho_n$) to generate multiple representations for a candidate content. Intuitively, we believe that multiple embeddings can enhance the user-candidate interactions when calculating the relevance score s_j^i between user u_i and candidate content $\bar{e}_j$. Similarly, we compute each candidate content vector ω_a using Eq. 4 with a trainable parameter W^o. The obtained n candidate content vectors are combined into $\Lambda \in \mathrm{R}^{n\times d}$.

3.4 Engagement Predictor

To compute the relevance score s_j^i, we first calculate the matching scores between the user representation embedding Γ_i and the candidate content representation embedding Λ_j by the inner product:

$$K_j^i = \mathrm{FLATTEN}(\Gamma_i^\top \Lambda_j). \tag{5}$$

The matrix is flattened into a $m \times n$ dimensional vector, denoted as $K_j^i \in \mathrm{R}^{mn}$. An attention layer is applied to aggregate these $m \times n$ matching scores:

$$\begin{aligned} s_j^i &= W^p K_j^i, \\ W^p &= \mathrm{softmax}\left[\mathrm{FLATTEN}\left[\Gamma \mathrm{gelu}(\Lambda W^s)^\top\right]\right], \end{aligned} \tag{6}$$

where $W^s \in \mathrm{R}^{d\times d}$ is a trainable parameter, $W^p \in \mathrm{R}^{mn}$ is the attention weights after flatten and softmax, and s_j^i is a scale of relevance score. We adhere to the common practice of training the model end-to-end using the noise contrastive estimation (NCE) loss (Wu et al., 2021a; Liu et al., 2023a).

4 Experiments

Dataset. We utilize two public datasets for content-based recommendation. The first is MIND dataset (Wu et al., 2020), which comprises user behavior logs from Microsoft News. We employ the small version of MIND dataset. The second

dataset is a book recommendation dataset sourced from Goodreads (Wan and McAuley, 2018), where user behaviors are inferred from book ratings. We provide more details and statistics of these datasets in Section A in Appendix.

Baselines. We compare our SPAR with several widely used and SoTA neural network-based content recommendation methods. These include methods that train text encoders from scratch, such as (1) NAML (Wu et al., 2019a), (2) NRMS (Wu et al., 2019c), (3) Fastformer (Wu et al., 2021b), (4) CAUM (Qi et al., 2022), and (5) MINS (Wang et al., 2022), as well as systems that leverage PLMs, including (6) NAML-PLM, (7) UNBERT (Zhang et al., 2021), (8) MINER (Li et al., 2022), and (9) UniTRec (Mao et al., 2023). For additional details on these baselines and their implementations, refer to Section B in Appendix.

Metrics. We adopt diverse metrics to evaluate content-based recommendation systems. These include the classification-based metric AUC (Fawcett, 2006), ranking-based metrics MRR (Voorhees, 1999) and nDCG@top$N$ (with top$N$ = 5, 10) (Järvelin and Kekäläinen, 2002). The Python library TorchMetrics (Detlefsen et al., 2022) is employed for metric calculations. We use AUC to determine the best model on Dev set and report Test performance on all the metrics.

Generating User-Interest Summary via LLM. As introduced in Section 3.2, we utilize LLM as a user-interest profiler to capture global user interests. In our experiments, we employ an open-source conversational model, LLaMA2-Chat-70B (Touvron et al., 2023b), to generate concise summaries reflecting users' engagement histories. The methodology for generating user-interest summaries is elaborated in Section C in Appendix.

Implementation and Hyperparameters. We utilize the pretrained RoBERTa-base model (Liu et al., 2019) as our content encoder. We perform hyperparameter tuning on the learning rate, the sizes of user-side and candidate-side codebooks (i.e., the UHS and CCS layers), and the local window size in the sparse attention mechanism of the UHS layers. The optimal codebook size of CCS layers is 4 and the optimal local attention window size is 512 for MIND dataset. For the Goodreads dataset, these are 4 and 256, respectively. The negative sampling ratios for the MIND and Goodreads datasets are 4 and 2, respectively. For both datasets, we incorporated the latest 60 user engagement contents as the user's history. The dimension size for both user and item representations is set at 200 across all experiments. Further details about hyperparameters are in Section D of Appendix.

	AUC	MRR	nDCG@5	nDCG@10
MIND-small				
NAML	66.10	34.65	32.80	39.14
NRMS	63.28	33.10	31.50	37.68
Fastformer	66.32	34.75	33.03	39.30
CAUM	62.56	34.40	32.88	38.90
MINS	61.43	35.99	34.13	40.54
NAML-PLM	67.01	35.67	34.10	40.32
UNBERT	71.73	38.06	36.67	42.92
MINER	70.20	38.10	36.35	42.63
UniTRec	69.38	37.62	36.01	42.20
SPAR (ours)	**73.21**	**39.51**	**37.80**	**44.01**
Goodreads				
NAML	59.35	72.16	53.49	67.81
NRMS	60.51	72.15	53.69	68.03
Fastformer	59.39	71.11	52.38	67.05
CAUM	55.13	73.06	54.97	69.02
MINS	53.02	71.81	53.72	68.00
NAML-PLM	59.57	72.54	53.98	68.41
UNBERT	61.40	73.34	54.67	68.71
MINER	60.72	72.72	54.17	68.42
UniTRec	60.00	72.60	53.73	67.96
SPAR (ours)	**62.55**	**73.97**	**55.48**	**69.51**

Table 1: Comparison of Test performance. The best-performing results are highlighted in **bold** font. The second-best performing model is underscored.

5 Results

5.1 Overall Results

Table 1 presents the Test results for models with median performance across three runs using different seeds. NAML-PLM (Wu et al., 2019c), UNBERT (Zhang et al., 2021), MINER (Li et al., 2022) and UniTRec (Mao et al., 2023), which employ PLMs for content encoding, demonstrate a significant advantage over models that solely depend on pretrained word embeddings. For instance, we observe that NAML achieves 0.91 AUC and 0.22 AUC improvement on MIND and Goodreads datasets, respectively, when a PLM is used as the content encoder instead of training an encoder from scratch. This superiority is observed consistently across both the MIND and Goodreads datasets, underscoring the efficacy of using advanced LMs in content-based recommendation systems. Our proposed SPAR achieves the highest AUC scores on both datasets, recording a remarkable 73.21 for MIND and 62.55 for Goodreads. When compared to UNBERT, our SPAR exhibits a significant in-

crease of 1.48 in AUC for MIND (t-test on AUC, $p < 0.02$) and 1.15 for Goodreads (t-test on AUC, $p < 0.05$). Beyond AUC, our framework attains the best performance across all other ranking-based metrics for both datasets. We also provide the mean and standard deviation across three runs in Tables 6 and 7 in Appendix, respectively.

5.2 Ablation Studies and Analyses

Ablation Studies. To better understand the effectiveness of our framework, we conduct ablation studies on MIND dataset, the results of which are presented in Table 2. We first remove the UHS layer from SPAR (row d in Table 2) and use the encoder output representation of '$\langle SOS \rangle$' for each piece of user historical content.[7] This alteration leads to a significant performance decrease of 1.06 in AUC, underscoring the importance of the UHS layer in summarizing user history. We then retain this UHS layer but replace our proposed sparse attention mechanism with full attention applied to the entire sequences of user engaged contents (row e). This change results in a performance drop of 0.51 in AUC, further highlighting the utility of sparse attention in utilizing the UHS layer. Following (Han et al., 2023), we empirically investigate the attention weights within the UHS layer. By processing samples from Dev set through each model, we extract the attention weights for the first three codes in the UHS layer. We then calculate the entropy of the attention distribution for each code per sample, averaging across the three poly-attention codes and all samples. Our findings indicate a decrease in entropy from 7.74 to 5.66 upon implementing sparse attention in the UHS layer.

We explore the impact of removing the UIE layer by setting the query codebook size to 1 (row f), which also results in a performance decrease of 0.16 in AUC, indicating that a single user embedding is insufficient. We then experiment with newly initializing the Transformer encoder layers and training them end-to-end (row a), which leads to a significant performance drop of 5.76 in AUC, demonstrating the crucial role of using a PLM. We also keep the PLM frozen and only train the new layers (row b), which results in a performance drop of 2.86 AUC. After removing session-based history content grouping (row g), the LLM-generated user-interest summary (row h), and the CCS layer (row i), we observe slight performance drops of 0.07, 0.06, and 0.02 in AUC, respectively. We then also remove these three components together (row j), but this leads to a large performance drop of 0.46 AUC. These findings suggest that while these components individually contribute to smaller performance enhancements, collectively, they are integral to the overall effectiveness of our framework.

[7]Note that the sparse attention is also removed with this UHS layer.

	AUC	MRR	nD@5	nD@10
SPAR	**73.21**	**39.51**	37.80	44.01
(a) wo PLM	66.45	32.33	31.22	37.47
(b) wo original history	70.37	37.46	35.91	42.19
(c) freeze PLM	70.85	38.75	37.21	43.17
(d) wo UHS	72.15	38.71	37.38	43.56
(e) wo attention sparsity	72.70	39.10	37.36	43.65
(f) wo UIE	73.05	39.43	37.84	44.05
(g) wo session grouping	73.14	39.50	37.79	43.92
(h) wo LLM sum.	73.15	39.40	37.61	44.01
(i) wo CCS	73.19	39.46	**38.06**	**44.07**
(j) wo (g), (h), (i)	72.75	39.39	37.55	43.87

Table 2: Test performance of ablation studies on MIND dataset. **nD@**N represents nDCG@N.

We also experiment with using only the LLM-generated user-interest summary as the user-side input and setting the codebook size of the UHS layer to 1, considering that the user side consists of a single history content (i.e., user-interest summary). This setting (row b) results in an AUC of 70.37 and an MRR of 37.46, significantly underperforming SPAR. This indicates the importance of using the original user history for learning user interests.

	AUC	MRR	nDCG@5	nDCG@10
SPAR	**73.21**	**39.51**	**37.80**	**44.01**
wo UHS	72.15	38.71	37.38	43.56
S-Longf.	73.02	39.34	38.11	43.92
S-BigB.	71.17	38.13	36.82	43.05

Table 3: Comparing different backbone models.

Handling Long Sequences. As the aforementioned results indicate, managing long sequences of user engagement history is a crucial component. We explore whether using PLM for processing long sequences could be a superior solution compared to our proposed framework. Two long sequence PLMs are utilized in this experiment, namely, Longformer-base (Beltagy et al., 2020) and BigBird-base (Zaheer et al., 2020). These models are Transformer encoder-only models, pretrained for processing up to 4,096 tokens. We leverage these long sequence PLMs as a content encoder

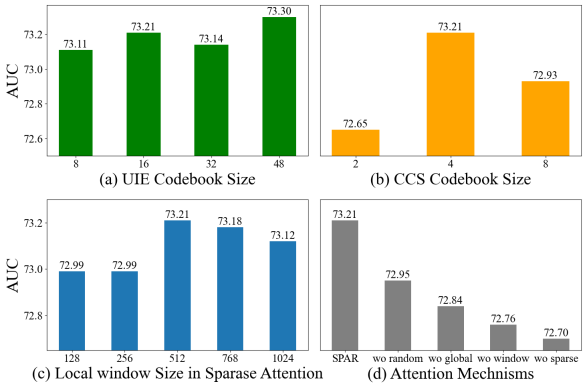

Figure 2: Influence of different hyperparameters.

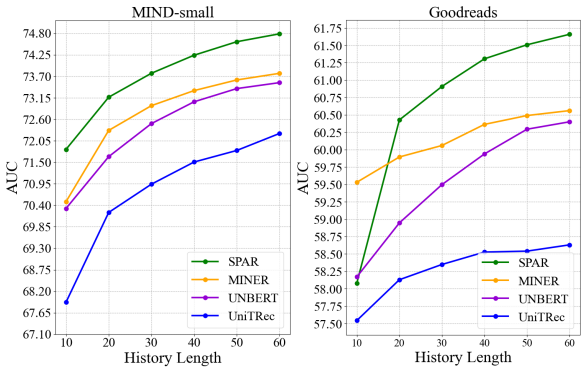

Figure 3: Effects of user engagement history length.

and remove the UHS layer from our framework. To ensure the encoding of all 60 historical contents of a given user, we divide user history into two groups, each potentially containing up to 30 user historical engagement contents.[8] We extract the encoder output representation of '$\langle SOS \rangle$' for each piece of user historical content and pass the sequence of user historical contents through the UIE layer in our framework. We refer to these two variants as SPAR-Longformer and SPAR-BigBird. As Table 3 illustrates, both SPAR-Longformer and SPAR-BigBird underperform our SPAR with a performance gap of 0.19 and 2.04 AUC, respectively. While SPAR-Longformer performs better than our ablation model with the UHS layer removed, it still lags behind SPAR.[9] This suggests the effectiveness of our proposed method in handling long user history with a post-fusion strategy.

Influence of Hyperparameters. Figure 2 illustrates the effects of various hyperparameters. Figure 2a shows that generally an increase in UIE codebook size correlates with higher AUC scores. A UIE codebook size of 16 is preferred to minimize storage requirements, albeit at the cost of a marginal performance decline. Specifically, a UIE codebook size of 16 results in a 0.09 decrease in AUC but achieves a threefold reduction in storage space compared to a size of 48. Regarding the codebook size of the CCS layer (Figure 2b), our findings suggest that an increased number of codebooks on the candidate item side does not enhance model performance. We hypothesize that excessively numerous codebooks for a single item might introduce superfluous parameters, thereby detrimentally impacting performance. Figure 2c indicates that the optimal sliding window size for sparse attention in the UHS is 512, with longer attention spans (exceeding 512) in UHS codebook-based attention adversely affecting model efficacy. In Figure 2d, our analysis of various sparse attention mechanisms in the UHS layer reveals performance decrements of 0.26, 0.37, and 0.45 in AUC upon the removal of random, global, and local window attention mechanisms, respectively. These findings underscore the significance of incorporating both local window and global attention mechanisms in the UHS layer.

Effects of User Engagement History Length. We examine the impact of the quantity of user-engaged history on model performance. We utilize a subset of Test users who are not present in the Train set and possess a minimum of 60 engaged contents. In cases where a user has multiple impressions, we randomly select one impression to represent that user. The user histories are truncated to the K most recent contents, with $K = \{10, 20, 30, 40, 50, 60\}$, while maintaining the same candidate contents in the impression. As shown in Figure 3, SPAR outperforms the three strong baselines across all engagement history lengths in MIND dataset. Notably, while SPAR initially trails behind MINER and UNBERT with only 10 historical entries, it demonstrates marked improvement as the number of user-engaged histories increases. These findings underscore the effectiveness of extensive user histories and highlight the capability of SPAR in processing lengthy sequences for content-based recommendation. We provide results in other ranking metrics in Tables 9, 10, and 11 in Appendix.

Performance on Old and New Users. Test users are divided into two categories: old users, who are present in the Train set, and new users, who have not been previously included in the Train set. Ta-

[8] A single sequence of 60 user history contents exceeds the maximal sequence length in some cases.

[9] When we use eight Nvidia A100 40G GPUs, training SPAR-Longformer is also notably slower, requiring 8.5 hours per epoch, whereas SPAR completes in 3 hours per epoch.

		MIND		Goodreads	
		AUC	nD@5	AUC	nD@5
Old User	UNBERT	71.65	36.98	61.27	54.82
	MINER	70.37	36.41	60.76	54.92
	UniTRec	69.87	36.93	60.08	53.94
	SPAR	**73.29**	**38.08**	**62.67**	**55.42**
New User	UNBERT	71.75	36.95	60.94	54.40
	MINER	70.18	36.28	60.59	54.20
	UniTRec	69.85	36.66	59.37	52.78
	SPAR	**73.19**	**37.74**	**62.51**	**55.51**

Table 4: Comparing performance on old and new users. The new users are users that do not occur in Train set. nD@5: nDCG@5.

ble 4 presents a comparison of model performance between these two user groups. It is observed that our SPAR surpasses all baselines in both groups across two datasets. Generally, the performance of the old user group is slightly better than that of the new user group. For instance, the performance gap between the old and new user groups in our SPAR is 0.10 in AUC. This outcome suggests the effective transferability of content-based recommendation methods to new users.

6 Conclusion

We introduced a novel framework, SPAR, designed to obtain independent representations for both users and candidate content. Our SPAR excels at extracting fine-grained features from long user engagement histories, significantly enhancing the post-interaction dynamics between users and candidate contents. By integrating a poly-attention scheme with a sparse attention mechanism, SPAR effectively aggregates lengthy user history sequences while maintaining relatively low computational costs. Through our experiments on two benchmark datasets, we have demonstrated that our framework achieves SoTA performance. Our ablation studies and model analyses have demonstrated the effectiveness of each component in our proposed framework and highlighted its robustness for content-based recommendation.

7 Limitations

In this study, we exclusively focus on content-based recommendation, with text features as the only input. We acknowledge that real-world recommendation systems require non-content related sparse/dense features or content features from other modalities. While our framework is tailored for text content, we envision it as an effective component contributing to the overall recommendation system.

In our experiments, we opted for a base-sized encoder model to strike a balance between computational efficiency and model performance. However, for practical real-world applications, smaller-sized models are preferable due to their faster inference speed. In future work, we plan to investigate the use of other smaller-sized models as the backbone, aiming to optimize the trade-off between performance and efficiency in real-world scenarios.

8 Ethical Considerations

We employed an LLM to generate summaries of user interests, which serve as input for learning user-side representations. It is critical to recognize that the LLM's outputs may reflect societal biases (Lucy and Bamman, 2021) or produce inaccuracies known as hallucinations (Zhang et al., 2023). Consequently, the recommendation models developed using these summaries might also exhibit such biases. However, we expect that the active research aimed at enhancing the social fairness, accuracy, and reliability of LLMs (Dev et al., 2022; Ji et al., 2023; Zhou et al., 2023) will also improve the performance and ethical standards of recommendation systems incorporating LLMs as a component.

In our experiments, we utilized two publicly available datasets designated for research purposes. The datasets' original authors (Wu et al., 2020; Wan and McAuley, 2018) have anonymized user identities to protect privacy.

References

Mingxiao An, Fangzhao Wu, Chuhan Wu, Kun Zhang, Zheng Liu, and Xing Xie. 2019. Neural news recommendation with long- and short-term user representations. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 336–345, Florence, Italy. Association for Computational Linguistics.

Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Anja Hauth, Katie Millican, David Silver, Slav Petrov, Melvin Johnson, Ioannis Antonoglou, Julian Schrittwieser, Amelia Glaese, Jilin Chen, Emily Pitler, Timothy P. Lillicrap, Angeliki Lazaridou, Orhan Firat, James Molloy, Michael Isard, Paul Ronald Barham, Tom Hennigan, Benjamin Lee, Fabio Viola, Malcolm Reynolds, Yuanzhong Xu, Ryan Doherty, Eli Collins, Clemens Meyer, Eliza Rutherford, Erica Moreira, Kareem

Ayoub, Megha Goel, George Tucker, Enrique Piqueras, Maxim Krikun, Iain Barr, Nikolay Savinov, Ivo Danihelka, Becca Roelofs, Anaïs White, Anders Andreassen, Tamara von Glehn, Lakshman Yagati, Mehran Kazemi, Lucas Gonzalez, Misha Khalman, Jakub Sygnowski, and et al. 2023. Gemini: A family of highly capable multimodal models. *CoRR*, abs/2312.11805.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer.

Qiwei Bi, Jian Li, Lifeng Shang, Xin Jiang, Qun Liu, and Hanfang Yang. 2022. MTRec: Multi-task learning over BERT for news recommendation. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 2663–2669, Dublin, Ireland. Association for Computational Linguistics.

Zeyu Cui, Jianxin Ma, Chang Zhou, Jingren Zhou, and Hongxia Yang. 2022. M6-Rec: Generative Pretrained Language Models are Open-Ended Recommender Systems.

Nicki Skafte Detlefsen, Jirí Borovec, Justus Schock, Ananya Harsh Jha, Teddy Koker, Luca Di Liello, Daniel Stancl, Changsheng Quan, Maxim Grechkin, and William Falcon. 2022. Torchmetrics - measuring reproducibility in pytorch. *J. Open Source Softw.*, 7(69):4101.

Sunipa Dev, Emily Sheng, Jieyu Zhao, Aubrie Amstutz, Jiao Sun, Yu Hou, Mattie Sanseverino, Jiin Kim, Akihiro Nishi, Nanyun Peng, and Kai-Wei Chang. 2022. On measures of biases and harms in NLP. In *Findings of the Association for Computational Linguistics: AACL-IJCNLP 2022*, pages 246–267, Online only. Association for Computational Linguistics.

Tom Fawcett. 2006. An introduction to ROC analysis. *Pattern Recognit. Lett.*, 27(8):861–874.

Hongyu Gong, Yelong Shen, Dian Yu, Jianshu Chen, and Dong Yu. 2020. Recurrent chunking mechanisms for long-text machine reading comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 6751–6761, Online. Association for Computational Linguistics.

Youyang Gu, Tao Lei, Regina Barzilay, and Tommi Jaakkola. 2016. Learning to refine text based recommendations. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2103–2108, Austin, Texas. Association for Computational Linguistics.

Chi Han, Qifan Wang, Wenhan Xiong, Yu Chen, Heng Ji, and Sinong Wang. 2023. LM-Infinite: Simple On-the-Fly Length Generalization for Large Language Models.

Ruining He and Julian J. McAuley. 2016. Fusing similarity models with markov chains for sparse sequential recommendation. In *IEEE 16th International Conference on Data Mining, ICDM 2016, December 12-15, 2016, Barcelona, Spain*, pages 191–200. IEEE Computer Society.

Samuel Humeau, Kurt Shuster, Marie-Anne Lachaux, and Jason Weston. 2019. Poly-encoders: Transformer Architectures and Pre-training Strategies for Fast and Accurate Multi-sentence Scoring.

Andreea Iana, Goran Glavaš, and Heiko Paulheim. 2023. NewsRecLib: A PyTorch-Lightning Library for Neural News Recommendation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 296–310, Singapore. Association for Computational Linguistics.

Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 874–880, Online. Association for Computational Linguistics.

Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated gain-based evaluation of IR techniques. *ACM Trans. Inf. Syst.*, 20(4):422–446.

Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. 2023. Towards mitigating LLM hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 1827–1843, Singapore. Association for Computational Linguistics.

Xuan Nhat Lam, Thuc Vu, Trong Duc Le, and Anh Duc Duong. 2008. Addressing cold-start problem in recommendation systems. In *Proceedings of the 2nd International Conference on Ubiquitous Information Management and Communication, ICUIMC 2008, Suwon, Korea, January 31 - February 01, 2008*, pages 208–211. ACM.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, Online. Association for Computational Linguistics.

Jian Li, Jieming Zhu, Qiwei Bi, Guohao Cai, Lifeng Shang, Zhenhua Dong, Xin Jiang, and Qun Liu. 2022. MINER: Multi-interest matching network for news recommendation. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 343–352, Dublin, Ireland. Association for Computational Linguistics.

Qijiong Liu. 2023. Legommenders: A modular framework for recommender systems.

Qijiong Liu, Nuo Chen, Tetsuya Sakai, and Xiao-Ming Wu. 2023a. ONCE: Boosting Content-based Recommendation with Both Open- and Closed-source Large Language Models.

Qijiong Liu, Jieming Zhu, Quanyu Dai, and Xiao-Ming Wu. 2022. Boosting deep CTR prediction with a plug-and-play pre-trainer for news recommendation. In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 2823–2833, Gyeongju, Republic of Korea. International Committee on Computational Linguistics.

Qijiong Liu, Jieming Zhu, Quanyu Dai, and Xiao-Ming Wu. 2023b. Only Encode Once: Making Content-based News Recommender Greener.

Rui Liu, Bin Yin, Ziyi Cao, Qianchen Xia, Yong Chen, and Dell Zhang. 2023c. PerCoNet: News Recommendation with Explicit Persona and Contrastive Learning.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized BERT pretraining approach. *ArXiv preprint*, abs/1907.11692.

Zhenghao Liu, Sen Mei, Chenyan Xiong, Xiaohua Li, Shi Yu, Zhiyuan Liu, Yu Gu, and Ge Yu. 2023d. Text Matching Improves Sequential Recommendation by Reducing Popularity Biases.

Li Lucy and David Bamman. 2021. Gender and representation bias in GPT-3 generated stories. In *Proceedings of the Third Workshop on Narrative Understanding*, pages 48–55, Virtual. Association for Computational Linguistics.

Chenyang Lyu, Minghao Wu, Longyue Wang, Xinting Huang, Bingshuai Liu, Zefeng Du, Shuming Shi, and Zhaopeng Tu. 2023. Macaw-llm: Multi-modal language modeling with image, audio, video, and text integration. *CoRR*, abs/2306.09093.

Itzik Malkiel, Oren Barkan, Avi Caciularu, Noam Razin, Ori Katz, and Noam Koenigstein. 2020. RecoBERT: A catalog language model for text-based recommendations. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1704–1714, Online. Association for Computational Linguistics.

Zhiming Mao, Huimin Wang, Yiming Du, and Kam-Fai Wong. 2023. UniTRec: A unified text-to-text transformer and joint contrastive learning framework for text-based recommendation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1160–1170, Toronto, Canada. Association for Computational Linguistics.

Shumpei Okura, Yukihiro Tagami, Shingo Ono, and Akira Tajima. 2017. Embedding-based news recommendation for millions of users. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Halifax, NS, Canada, August 13 - 17, 2017*, pages 1933–1942. ACM.

OpenAI. 2023. GPT-4 technical report. *CoRR*, abs/2303.08774.

Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, Doha, Qatar. Association for Computational Linguistics.

Tao Qi, Fangzhao Wu, Chuhan Wu, and Yongfeng Huang. 2022. News recommendation with candidate-aware user modeling. In *SIGIR '22: The 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, Madrid, Spain, July 11 - 15, 2022*, pages 1917–1921. ACM.

Tao Qi, Fangzhao Wu, Chuhan Wu, Peiru Yang, Yang Yu, Xing Xie, and Yongfeng Huang. 2021. HieRec: Hierarchical user interest modeling for personalized news recommendation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 5446–5456, Online. Association for Computational Linguistics.

Steffen Rendle. 2010. Factorization machines. In *ICDM 2010, The 10th IEEE International Conference on Data Mining, Sydney, Australia, 14-17 December 2010*, pages 995–1000. IEEE Computer Society.

Andrew I. Schein, Alexandrin Popescul, Lyle H. Ungar, and David M. Pennock. 2002. Methods and metrics for cold-start recommendations. In *SIGIR 2002: Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, August 11-15, 2002, Tampere, Finland*, pages 253–260. ACM.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023a. Llama: Open and efficient foundation language models. *CoRR*, abs/2302.13971.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Moly-

bog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023b. Llama 2: Open foundation and fine-tuned chat models. *ArXiv preprint*, abs/2307.09288.

Ellen M. Voorhees. 1999. The TREC-8 question answering track report. In *Proceedings of The Eighth Text REtrieval Conference, TREC 1999, Gaithersburg, Maryland, USA, November 17-19, 1999*, volume 500-246 of *NIST Special Publication*. National Institute of Standards and Technology (NIST).

Mengting Wan and Julian J. McAuley. 2018. Item recommendation on monotonic behavior chains. In *Proceedings of the 12th ACM Conference on Recommender Systems, RecSys 2018, Vancouver, BC, Canada, October 2-7, 2018*, pages 86–94. ACM.

Rongyao Wang, Shoujin Wang, Wenpeng Lu, and Xueping Peng. 2022. News recommendation via multi-interest news sequence modelling. In *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2022, Virtual and Singapore, 23-27 May 2022*, pages 7942–7946. IEEE.

Yuling Wang, Changxin Tian, Binbin Hu, Yanhua Yu, Ziqi Liu, Zhiqiang Zhang, Jun Zhou, Liang Pang, and Xiao Wang. 2024. Can small language models be good reasoners for sequential recommendation? In *Proceedings of the ACM on Web Conference 2024, WWW 2024, Singapore, May 13-17, 2024*, pages 3876–3887. ACM.

Chuhan Wu, Fangzhao Wu, Mingxiao An, Jianqiang Huang, Yongfeng Huang, and Xing Xie. 2019a. Neural news recommendation with attentive multi-view learning. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*, pages 3863–3869. ijcai.org.

Chuhan Wu, Fangzhao Wu, Mingxiao An, Jianqiang Huang, Yongfeng Huang, and Xing Xie. 2019b. NPA: neural news recommendation with personalized attention. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4-8, 2019*, pages 2576–2584. ACM.

Chuhan Wu, Fangzhao Wu, Suyu Ge, Tao Qi, Yongfeng Huang, and Xing Xie. 2019c. Neural news recommendation with multi-head self-attention. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 6389–6394, Hong Kong, China. Association for Computational Linguistics.

Chuhan Wu, Fangzhao Wu, Tao Qi, and Yongfeng Huang. 2021a. Empowering News Recommendation with Pre-trained Language Models.

Chuhan Wu, Fangzhao Wu, Tao Qi, Yongfeng Huang, and Xing Xie. 2021b. Fastformer: Additive Attention Can Be All You Need.

Chuhan Wu, Fangzhao Wu, Yang Yu, Tao Qi, Yongfeng Huang, and Qi Liu. 2021c. NewsBERT: Distilling pre-trained language model for intelligent news application. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 3285–3295, Punta Cana, Dominican Republic. Association for Computational Linguistics.

Fangzhao Wu, Ying Qiao, Jiun-Hung Chen, Chuhan Wu, Tao Qi, Jianxun Lian, Danyang Liu, Xing Xie, Jianfeng Gao, Winnie Wu, and Ming Zhou. 2020. MIND: A large-scale dataset for news recommendation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 3597–3606, Online. Association for Computational Linguistics.

Minghao Wu and Alham Fikri Aji. 2023. Style over substance: Evaluation biases for large language models. *CoRR*, abs/2307.03025.

Minghao Wu, Abdul Waheed, Chiyu Zhang, Muhammad Abdul-Mageed, and Alham Fikri Aji. 2023. Lamini-lm: A diverse herd of distilled models from large-scale instructions. *CoRR*, abs/2304.14402.

Liancheng Xu, Xiaoxiang Wang, Lei Guo, Jinyu Zhang, Xiaoqi Wu, and Xinhua Wang. 2023. Candidate-aware dynamic representation for news recommendation. In *Artificial Neural Networks and Machine Learning - ICANN 2023 - 32nd International Conference on Artificial Neural Networks, Heraklion, Crete, Greece, September 26-29, 2023, Proceedings, Part VII*, volume 14260 of *Lecture Notes in Computer Science*, pages 272–284. Springer.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*.

Qi Zhang, Jingjie Li, Qinglin Jia, Chuyuan Wang, Jieming Zhu, Zhaowei Wang, and Xiuqiang He. 2021. UNBERT: User-News Matching BERT for News Recommendation. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*, pages 3356–3362, Montreal, Canada. International Joint Conferences on Artificial Intelligence Organization.

Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, Longyue Wang, Anh Tuan Luu, Wei

Bi, Freda Shi, and Shuming Shi. 2023. Siren's song in the AI ocean: A survey on hallucination in large language models. *ArXiv preprint*, abs/2309.01219.

Wenxuan Zhou, Sheng Zhang, Hoifung Poon, and Muhao Chen. 2023. Context-faithful prompting for large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 14544–14556, Singapore. Association for Computational Linguistics.

Appendices

A Dataset

We utilize two public benchmark datasets for content-based recommendation. The first is MIND dataset from Wu et al. (2020), which comprises user behavior logs from Microsoft News. This dataset includes positive and negative labels that indicate whether a user clicked on a news article in an impression. We employ the small version of the MIND dataset, containing 94,000 users and 65,238 news articles for training and validation. The validation set is divided into a 10% Dev set and a 90% Test set. Each news article in the MIND dataset is characterized by a title, abstract, and category label. Our second dataset is a book recommendation dataset sourced from Goodreads (Wan and McAuley, 2018), where user behaviors are inferred from book ratings. Positive labels signify ratings above 3, while negative labels indicate ratings below 3. Each book entry includes a name, author, description, and genre category. Table 5 details the data distribution and statistics.

B Baselines

We compare our proposed model with previous SoTA neural network-based methods for content-based recommendation:

(1) NAML (Wu et al., 2019a) trains a contents representation by using a sequence of CNN and additive attention and pretrained word embeddings. To get a user's embedding, additional additive attention is applied to the embeddings of the user's engaged contents.

(2) NRMS (Wu et al., 2019c) utilizes pretrained word embeddings, multi-head self-attention, and additive attention layers to learn representations for both users and candidate content.

(3) Fastformer (Wu et al., 2021b) is an efficient Transformer architecture based on additive attention.

(4) CAUM (Qi et al., 2022) enhances NRMS by incorporating title entities into content embeddings and introducing candidate-aware self-attention for user embedding.

(5) MINS (Wang et al., 2022) advances NRMS by implementing a multi-channel GRU-based recurrent network to learn sequential information in user history.

Dataset	MIND			Goodreads			Dataset	MIND	Goodreads
Split	Train	Dev	Test	Train	Dev	Test			
# content	51,283	21,352	41,496	309,047	234,232	247,242	# of history/user	22	47
# users	50,000	6,679	46,549	21,450	16,339	17,967	# category	18	11
# new users	-	5,862	41,020	-	2,930	3,199	# tokens/title	29	16
# positive	236,344	10,775	100,608	198,403	75,445	93,156	# tokens/abstract	144	190
# negative	5,607,100	249,607	2,380,008	458,435	141,977	154,016	# tokens/user summary	70	106

Table 5: Dataset Statistics. The row '# new users' indicates the number of users not included in the Train set. '# tokens/user summary' represents the average length of user interest summaries generated by LLM. The number of tokens are calculated using the RoBERTa-base model's vocabulary.

(6) NAML-PLM uses a PLM as a content encoder instead of a training encoder from scratch.

(7) UNBERT (Zhang et al., 2021) utilizes a PLM to encode input content and captures user-content matching signals at both item-level and word-level.

(8) MINER (Li et al., 2022) employs a PLM as a text encoder and adopts a poly attention scheme to extract multiple user interest vectors for user representation. MINER is arguably the most widely referred and top-ranked model in the MIND dataset leaderboard.[10]

(9) UniTRec (Mao et al., 2023) utilizes encoder-decoder architecture (i.e., BART) and encodes user history by encoder and candidate content by decoder respectively. They rank the candidate contents based on their perplexity from the decoder and discriminative scoring head.[11]

We adhere to the optimal hyperparameters of these SoTA baselines and train as well as evaluate them on our data splits. For NAML, NRMS, Fastformer, and NAML-PLM, we exploit the implementation by Liu (2023). For CAUM and MINS, we use the implementation provided by Iana et al. (2023). UNBERT, MINER, and UniTRec are implemented via the author-released scripts, respectively.[12]

[10] https://msnews.github.io/

[11] We use the predictions from discriminative scoring head to calculate metrics.

[12] UNBET: https://github.com/reczoo/RecZoo/tree/main/pretraining/news/UNBERT, MINER: https://github.com/duynguyen-0203/miner, UniTRec: https://github.com/Veason-silverbullet/UniTRec.

C Generating User-Interest Summary via LLM

We employ an open-source conversational LLM, specifically LLaMA2-Chat-70B (Touvron et al., 2023b), to generate summaries of user interests based on their engagement history. Figure 4 illustrates an example of our input and the generated output for the MIND dataset. Following the prompt template of LLaMA2-Chat models, we start with a system instruction including special tokens to establish the task context, followed by a list of user-browsed news history, sorted from the most recent to the oldest. Each listed news item includes its title, abstract, and category name. We limit the input prompt to 30 engaged contents and truncate longer news abstracts or book descriptions to 100 words to maintain sequence length within the maximal capacity of LLaMA2. The prompt concludes with a task-specific instruction requesting the model to summarize user interests in three sentences.[13] The lower section of Figure 4 displays a example generation from the LLM. As demonstrated in Table 5, the average length of these generated summaries is 70 tokens for the MIND dataset and 106 tokens for the Goodreads dataset.

D Implementation and Hyperparameters

We utilize the pretrained RoBERTa-base model (Liu et al., 2019) as our backbone model for encoding contents. We set the batch size to 128 and train the model for 10 epochs using the Adam optimizer, with a linearly decaying learning rate and a 10% warm-up phase. For the newly initialized layers, we employ a learning rate five times higher than the base learning rate. Hyperparameter tuning is performed within the following search space: base peak learning rate

[13] We manually fine-tuned this input template based on a few examples.

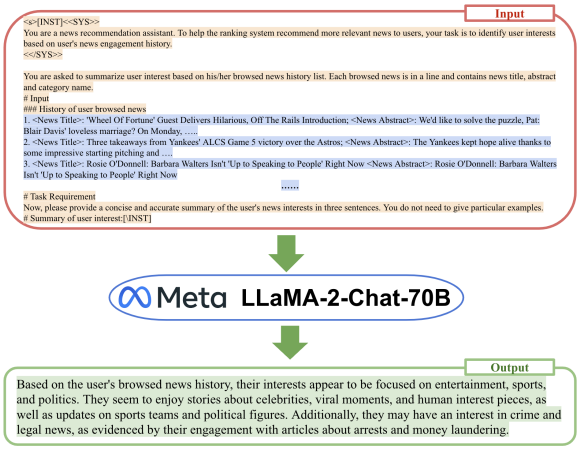

Figure 4: Example of prompting an LLM as a user profiler. The text within the red box represents the input to the LLM, while the text within the green box is the LLM's generated output. In the input text, the part highlighted in orange signifies the task instruction, and the text in blue denotes the user's browsed news history.

$= \{5e-5, 2e-5\}$, size of user-side context codes $= \{16, 32, 48\}$, candidate-side codebook size $= \{2, 4, 8\}$, and size of the local window in sparse attention $= \{256, 512, 1024\}$. For computational efficiency, 20% of the training data is randomly sampled for hyperparameter tuning, with the best parameters determined based on Dev set performance. The identified optimal hyperparameters for both datasets are a peak learning rate of $2e-5$ and a user-side context codebook size of 16. For the MIND dataset, the optimal candidate-side codebook size is 4 and the local attention window size is 512. For the Goodreads dataset, these are 4 and 256, respectively. For all experiments, we incorporated the most recent 60 user engagement contents as the user's history. In the MIND dataset, the negative sampling ratio is set to 4, the maximum length of a news title to 32 tokens, and the maximum length of a news abstract to 72 tokens. In the Goodreads dataset, the negative sampling ratio is 2, the maximum length of a book name is 24 tokens, and the maximum length of a book description is 85 tokens. The dimension size for both user and item representations is set at 200 across all experiments. Evaluations on the Dev set are conducted every 610 steps (one-third of an epoch) for the MIND dataset and every 520 steps for the Goodreads dataset, with the Test performance of the best model reported. All models are trained using eight Nvidia A100 40G GPUs.

E Results

	AUC	MRR	nDCG@5	nDCG@10
MIND-small				
NAML	65.42	34.44	32.57	38.90
NRMS	63.69	33.15	31.33	37.63
Fastformer	66.22	34.47	32.74	39.12
CAUM	62.78	34.52	33.09	39.22
MINS	61.00	35.58	33.86	40.23
NAML-PLM	67.11	35.57	34.27	40.41
UNBERT	71.69	38.10	36.71	42.92
MINER	70.23	38.24	36.55	42.82
UniTRec	69.48	37.76	36.19	42.35
SPAR (ours)	**73.20**	**39.34**	**37.77**	**44.04**
Goodreads				
NAML	59.18	72.05	53.59	67.90
NRMS	60.55	72.18	53.66	68.02
Fastformer	59.39	71.18	52.37	67.09
CAUM	55.03	73.31	55.03	69.08
MINS	53.03	71.92	53.59	67.98
NAML-PLM	59.43	72.52	53.78	68.38
UNBERT	61.37	73.38	54.64	68.70
MINER	60.69	72.87	54.32	68.51
UniTRec	59.64	72.52	53.51	67.86
SPAR (ours)	**62.43**	**73.87**	**55.28**	**69.42**

Table 6: Comparison of Test performance. The average scores over three runs are reported, with the best-performing results highlighted in **bold** and the second-best underscored.

Table 6 and 7 display the mean and standard deviation of the Test set performance for all baselines and SPAR.

Tables 8, 9, 10, and 11 present the comprehensive results from investigating the effects of user engagement history length on AUC, MRR, ndcg@5, and ndcg@10, respectively.

MIND-small	AUC	MRR	nDCG@5	nDCG@10
NAML	1.65	0.52	0.58	0.57
NRMS	1.12	1.02	1.04	0.91
Fastformer	0.57	0.67	0.67	0.57
CAUM	0.85	1.02	0.92	1.04
MINS	1.13	0.83	0.67	0.62
NAML-PLM	0.21	0.41	0.32	0.48
UNBERT	0.20	0.37	0.23	0.21
MINER	0.43	0.31	0.40	0.36
UniTRec	0.34	0.40	0.44	0.35
SPAR (ours)	0.10	0.15	0.20	0.07
Goodreads				
NAML	0.31	0.24	0.23	0.18
NRMS	0.06	0.16	0.10	0.09
Fastformer	0.09	0.25	0.17	0.14
CAUM	0.45	0.55	0.24	0.29
MINS	0.55	0.27	0.27	0.18
NAML-PLM	0.34	0.04	0.45	0.09
UNBERT	0.15	0.13	0.11	0.05
MINER	0.13	0.48	0.41	0.35
UniTRec	0.97	0.40	0.39	0.29
SPAR (ours)	0.29	0.41	0.36	0.35

Table 7: Standard deviation of Test performance over three runs.

History leng.	10	20	30	40	50	60
MIND						
UNBERT	70.32	71.65	72.49	73.05	73.39	73.54
MINER	70.49	72.32	72.95	73.34	73.61	73.78
UniTRec	67.92	70.22	70.94	71.51	71.80	72.24
SPAR	**71.83**	**73.17**	**73.78**	**74.24**	**74.59**	**74.79**
Goodreads						
UNBERT	58.17	58.95	59.50	59.94	60.29	60.40
MINER	**59.53**	59.89	60.06	60.36	60.49	60.56
UniTRec	57.54	58.13	58.35	58.53	58.54	58.63
SPAR	58.08	**60.43**	**60.91**	**61.31**	**61.51**	**61.66**

Table 8: Complete AUC results from investigating the effects of user engagement history length.

History leng.	10	20	30	40	50	60
MIND						
UNBERT	37.20	38.45	39.49	39.66	40.53	40.30
MINER	38.56	39.81	40.14	40.35	40.30	40.65
UniTRec	37.55	39.01	39.52	39.69	40.13	40.28
SPAR	**38.58**	**39.88**	**40.78**	**40.93**	**41.61**	**41.42**
Goodreads						
UNBERT	73.93	74.16	73.68	74.93	75.55	75.21
MINER	**73.98**	**74.42**	**74.72**	**75.42**	74.74	74.64
UniTRec	72.84	73.28	72.88	73.68	73.67	73.46
SPAR	72.04	73.43	74.62	75.39	**75.71**	**75.20**

Table 9: Complete MRR results from investigating the effects of user engagement history length.

History leng.	10	20	30	40	50	60
MIND						
UNBERT	32.14	33.45	34.38	34.51	35.30	35.08
MINER	33.27	34.47	34.71	34.99	35.03	35.24
UniTRec	31.96	33.44	33.90	34.37	34.72	35.07
SPAR	**33.58**	**34.81**	**35.46**	**35.72**	**36.22**	**36.16**
Goodreads						
UNBERT	52.93	53.68	53.97	54.16	54.96	55.18
MINER	**54.12**	**54.21**	54.45	54.72	54.68	54.31
UniTRec	51.70	52.41	52.47	52.97	53.08	53.02
SPAR	51.57	54.10	**54.93**	**55.05**	**55.31**	**55.30**

Table 10: Complete ndcg@5 results from investigating the effects of user engagement history length.

History leng.	10	20	30	40	50	60
MIND						
UNBERT	38.63	39.94	40.61	41.05	41.69	41.86
MINER	39.63	40.91	41.39	41.57	41.72	41.86
UniTRec	38.23	40.03	40.57	41.00	41.36	41.77
SPAR	**40.19**	**41.20**	**42.00**	**42.33**	**42.73**	**42.77**
Goodreads						
UNBERT	66.39	66.87	67.2	67.78	68.19	67.78
MINER	**67.34**	67.61	67.67	67.93	67.71	67.70
UniTRec	65.84	66.11	66.04	66.29	66.24	66.28
SPAR	65.56	**67.62**	**68.29**	**68.68**	**68.67**	**68.75**

Table 11: Complete ndcg@10 results from investigating the effects of user engagement history length.